

Scientific programming, or in broader terms, scientific computing, deals with solving scientific problems with the help of computers, so as to obtain results more quickly and accurately. Computers have long been used for solving complex scientific problems—however, advancements in computer science and hardware technologies over the years have also allowed students and academicians to play around with robust scientific computation tools.

Although tools like Mathematica and Matlab remain commercial, the open source community has also developed some equally powerful computational tools, which can be easily used by students and independent researchers. In fact, these tools are so robust that they are now also used at educational institutions and research labs across the globe.

So, let's move on to setting up a scientific environment.

Setting up the environment

Most UNIX system/Linux distributions have Python installed by default. We will use Python 2.6.6 for the purposes of this article. It's recommended to install IPython, as it offers enhanced introspection, additional shell syntax, syntax highlighting and tab-completion. You can directly install IPython from Sourceforge, or you can download the bleeding-edge version from the GitHub

repository, <https://github.com/ipython/ipython.git>.

Next, we'll install the two most basic scientific computational packages for Python: NumPy and SciPy. The former is the fundamental package needed for scientific computing with Python. It contains a powerful N-dimensional array object, sophisticated functions, tools for integrating C/C++ and Fortran code with useful linear algebra, Fourier transforms, and random-number capabilities. The SciPy library is built to work with NumPy arrays, and provides many user-friendly and efficient numerical routines for numerical integration and optimisation.

Open the Synaptic Package Manager and install the *python-numpy* and *python-sciPy* packages. Now that we have NumPy and SciPy installed, let's get our hands dirty with some mathematical functions and equations!

Numerical computations with NumPy, SciPy and Maxima

NumPy offers efficient array computations with fixed-size, homogeneous, multi-dimensional array types, and a plethora of functions to perform various array operations.



Figure 1: Installing NumPy and SciPy

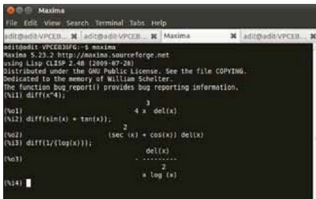


Figure 2: Differentiation of some simple functions

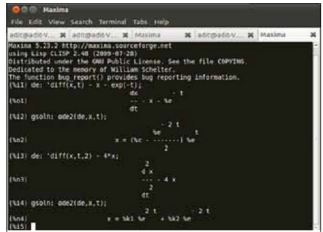


Figure 3: Solving simple differential equations

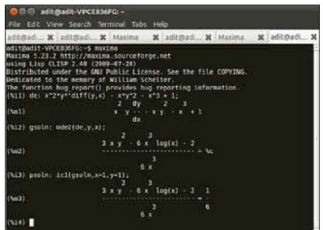


Figure 4: Getting solutions at a point of differential equations